

Session 1

De la Commande à l'Automatisation

Du terminal interactif au script intelligent.

Notre Mission du Jour

Aujourd'hui, nous allons construire la **compétence fondamentale** de tout DevOps :

1. **Diagnostiquer un système** avec des outils de précision.

- *Permissions, Processus, Logs*

2. **Transformer ce diagnostic en une action automatisée** via un script.

- *Conditions, Arguments, Bonnes pratiques*

Chaque partie se terminera par un atelier pratique pour solidifier les acquis.

Partie 1

Chapitre 1 : Les Permissions

Le Problème Quotidien :

*"J'ai créé mon script, je suis sûr qu'il est correct, mais quand je le lance, j'ai une erreur : **Permission denied**."*

La Cause :

Linux est un système sécurisé par défaut. Un nouveau fichier n'a pas le droit d'être **exécuté**. Il faut le lui accorder explicitement.

Théorie : Décoder les Droits avec `ls -l`

La commande `ls -l` révèle l'ADN d'un fichier.

```
d rwx r-x r-x
```

Type : `-` = fichier, `d` = dossier

Autres : Peuvent (`r`)

Groupe : Peuvent (`r`) (`x`)

Propriétaire (vous) : Droits complets (`rwx`)

L'Outil : `chmod` (Change Mode)

`chmod` est la commande pour modifier ces permissions.

La méthode simple et efficace (symbolique) :

- `chmod u+x script.sh` : Ajoute (`+`) la permission d'exécution (`x`) pour l'utilisateur (`u`).
- `chmod g-w config.txt` : Retire (`-`) le droit d'écriture (`w`) pour le groupe (`g`).

Pourquoi c'est important ?

Pour protéger des fichiers de configuration sensibles et autoriser l'exécution de vos outils.

La Voie du Pro : La Notation Octale

En entreprise, vous verrez très souvent des chiffres. C'est plus rapide.

- **r** = 4
- **w** = 2
- **x** = 1

On additionne pour chaque catégorie (Propriétaire, Groupe, Autres).

- **chmod 755 mon_script.sh** (**rwX r-X r-X**)

Le standard pour un script exécutable par tous.

- **chmod 644 mon_fichier.txt** (**rw- r-- r--**)

Le standard pour un fichier de données lisible par tous.

Atelier 1.1 : Débloquer son Script

Objectif : Résoudre un `Permission denied`.

1. Lancez votre labo : [Play with Docker](#) → `+ ADD NEW INSTANCE`.

2. Créez et provoquez l'erreur :

```
echo 'echo "Le script fonctionne !"' > go.sh  
./go.sh  
# ERREUR attendue !
```

3. Résolvez le problème avec `chmod` :

```
chmod +x go.sh  
./go.sh  
# SUCCÈS !
```

Chapitre 2 : Diagnostic Système

Le Problème Quotidien :

"Le site est lent. Une alerte dit que le CPU est à 100%. Comment trouver le coupable ?"

La Stratégie :

On ne devine pas. On utilise des outils pour **observer** le système et **filtrer** le bruit.

Les Outils d'Observation

- `ps aux` : Une **photographie** de tous les processus. Utile pour une analyse post-mortem.
- `top` : Une **vidéo** en temps réel des processus. Indispensable pour voir "qui" consomme en ce moment même. (`q` pour quitter).
- `kill <PID>` : Pour **arrêter** un processus proprement. Le `PID` est son numéro d'identification, visible avec `ps` ou `top`.

L'outil N°1 pour filtrer le bruit : `grep`

`grep` est une passoire. Il ne laisse passer que les lignes qui vous intéressent.

`ps aux | grep "nginx"` → Affiche uniquement les processus contenant le mot "nginx".

Atelier 1.2 : Le Détective de Processus

Objectif : Utiliser `grep` pour trouver et arrêter un processus spécifique.

1. Lancez un processus "suspect" en arrière-plan (`&`) :

```
sleep 300 &
```

2. Utilisez `ps` et `grep` pour l'identifier :

```
ps aux | grep "sleep"
```

Notez le numéro dans la 2ème colonne. C'est le `PID`.

3. Arrêtez-le proprement :

```
kill <VOTRE_PID>
```

4. Vérifiez qu'il a disparu (la même commande ne doit rien renvoyer) :

```
ps aux | grep "sleep"
```

Partie 2

Du Manuel à l'Automatique

La mentalité DevOps :

Si je dois faire une série d'actions plus de deux fois, j'écris un script.

Un script n'est pas juste une suite de commandes. C'est un **outil intelligent** qui peut s'adapter et prendre des décisions.

Théorie : L'Anatomie d'un Script Intelligent

1. Le Shebang : `#!/bin/bash`

Indique au système comment interpréter le fichier.

2. Les Variables : `SERVICE_NAME="nginx"`

Pour stocker de l'information et rendre le script facile à modifier.

3. La Logique Conditionnelle : `if [ ... ]; then ... fi`

Le cerveau du script. C'est ce qui lui permet de réagir.

4. Les Arguments : `$1`, `$2` ...

Pour rendre le script flexible et réutilisable.

Théorie : La Puissance du `if`

La syntaxe `if` permet de tester une condition.

Syntaxe de base :

```
if [ condition ]; then
    # Bloc exécuté si la condition est VRAIE
else
    # Bloc exécuté si la condition est FAUSSE
fi
```

Tests les plus courants :

- `[ -f "fichier.txt" ]` : Vrai si le fichier existe.
- `[ -d "/tmp/mondossier" ]` : Vrai si le dossier existe.
- `[ $COUNT -gt 0 ]` : Vrai si la variable `COUNT` est supérieure à (`-gt`) 0.

Atelier Final : Le Script de Surveillance Automatisé

Objectif : Créer un script réutilisable qui vérifie si n'importe quel service est en cours.

1. Lancez un faux service à surveiller : `sleep 600 &`
2. Créez et rendez exécutable le script `check_service.sh` :

```
touch check_service.sh && chmod +x check_service.sh
```

3. Ouvrez-le (`nano check_service.sh`) et collez ce code :

```
#!/bin/bash

# Le script attend un nom de service en argument.
SERVICE_NAME=$1

# Vérification : si l'argument est vide, on arrête avec une erreur.
if [ -z "$SERVICE_NAME" ]; then
    echo "Erreur : Vous devez spécifier un nom de service."
    exit 1 # Code de sortie 1 = Erreur
fi

# On compte les processus, en excluant notre propre commande grep.
# C'est une astuce de pro.
COUNT=$(ps aux | grep "$SERVICE_NAME" | grep -v "grep" | wc -l)

if [ $COUNT -gt 0 ]; then
    echo "✅ Le service '$SERVICE_NAME' est en cours d'exécution."
else
    echo "❌ ALERTE : Le service '$SERVICE_NAME' est arrêté !"
fi
```

4. Testez votre script intelligent :

```
./check_service.sh sleep # Test avec un service qui tourne  
./check_service.sh nginx # Test avec un service qui ne tourne pas  
./check_service.sh      # Test sans argument
```

Mission Accomplie

Aujourd'hui, nous avons relié deux mondes :

Le diagnostic manuel avec des outils précis.

L'automatisation intelligente avec des scripts qui s'adaptent.

C'est la compétence fondamentale sur laquelle repose tout le DevOps.

Questions ?